Machine Coding: Theme Toggle (Dark / Light / System)

- **Problem:** Add a theme switcher with three options: Light, Dark and System (follow the OS setting). The choice is saved and restored on reload **without a flash of the wrong theme**, the page follows OS changes live when on System, all colors come from CSS variables, native controls match the theme, open tabs stay in sync, and any component can read the theme through context.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS, `react-icons`)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	3 options, "system" default decided
Use cases + architecture + state	5 min	`preference` vs `resolvedTheme` split
CSS variables + `data-theme` + simple toggle	10 min	Working light/dark
Context + `useTheme` + persistence	10 min	Theme anywhere, survives reload
System option (`matchMedia`) + no-flash script	12 min	Correct on first paint
Cross-tab sync, a11y, `color-scheme`, polish	8 min	Production-quality details
Test (reload, OS switch, 2 tabs) + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"I separate what the user **chose** (light, dark or system) from what we **show** (light or dark). I store only the choice;*

the shown theme is derived from it and the OS setting. And the theme must be applied before React loads, or users with dark mode see a white flash on every reload."

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Three-way switch: Light / Dark / System. Default: System.
- Theme applies to the whole page (backgrounds, text, borders, buttons, links, inputs, scrollbars).
- The choice persists across reloads; no flash of the wrong theme on load.
- On System, the page follows OS changes live (e.g. macOS auto dark mode at sunset).
- Changing the theme in one tab updates other open tabs.
- Any component can read the current theme (`useTheme()`).

Non-functional requirements

Concern	What it means here
Performance	Switching = one attribute change on `<html>` ; CSS variables recolor everything without re-rendering components. The context value is memoised.
Scalability	Design tokens (`--bg` , `--text` , ...) instead of per-component colors; a new theme (high contrast) is one more CSS block.
Error handling	Invalid or missing stored value → System; blocked storage → works without saving.
Accessibility	A real radio group (arrow keys, "Dark, selected"), visible focus, icons `aria-hidden` , text labels, sufficient contrast in both themes, `prefers-reduced-motion` respected, `color-scheme` for native controls.
Reusability	`useTheme()` hook, `useSystemTheme()` hook, and pure theme utils.
Extensibility	More themes, per-component overrides, server-saved preferences, or SSR via a cookie.

Clarifying questions to ask the interviewer

1. Just light/dark, or also "System"? What's the default for a new user?
2. Should the choice persist? Per browser (localStorage) or per account (server)?
3. Is there server-side rendering (Next.js)? Then the flash problem needs a cookie or an inline script.
4. Should images/illustrations or charts have theme-specific variants?
5. Do we need more themes later (high contrast, brand themes)?
6. Animate the switch, or switch instantly?

Assumptions we will build with

- Options: light, dark, system; default system; stored in localStorage under `theme` as a plain string.
- Applied via `data-theme="light|dark"` on `<html>` , with all colors as CSS variables.
- An inline script in `index.html` applies the theme before the first paint.
- Live OS changes via `matchMedia("(prefers-color-scheme: dark)")` + `useSyncExternalStore` .
- Cross-tab sync with the `storage` event; a short color transition, off for reduced motion.

2. Identify interactions / use cases

Action / input	Expected behavior / output
First visit, OS in dark mode	Page loads dark immediately (no white flash); "System" selected
First visit, OS in light mode	Page loads light; "System" selected
Click "Dark"	Whole page switches to dark; saved
Click "Light"	Whole page switches to light; saved
Reload after choosing Light (OS dark)	Loads light immediately
On System, OS switches dark → light	Page switches live, no reload
On Light, OS switches	Nothing changes (explicit choice wins)
Stored value is garbage ("purple")	Treated as System
Storage blocked	Theme still works for this visit
Change theme in tab A	Tab B updates without reload
Keyboard	Tab into the group, arrow keys change the option, focus ring visible
Screen reader	"Theme, Dark, radio button, selected" + status text announced
Native input / scrollbar	Match the theme (`color-scheme`)
Reduced motion	Colors switch instantly

3. Design the architecture

Module	Responsibility
`index.html` inline script	Read the stored preference + OS setting and set `data-theme` **before** the first paint
`utils/theme.js`	`PREFERENCES` , `STORAGE_KEY` , `resolveTheme` , `readPreference` , `savePreference` , `applyTheme`
`hooks/useSystemTheme.js`	OS theme as React state via `useSyncExternalStore` + `matchMedia`
`context/themeContext.js`	`ThemeContext` (default `null`)
`context/ThemeProvider.jsx`	Owns `preference` ; derives `resolvedTheme` ; applies, saves and syncs across tabs
`hooks/useTheme.js`	Reads the context; throws if used outside the provider
`components/ThemeSwitcher.jsx`	Light / Dark / System radio group
`index.css`	Design tokens for both themes + `color-scheme`

```
index.html (inline script) ──sets──▶ <html data-theme="dark">   (before CSS paints)
main.jsx
└── <ThemeProvider>          preference (state) + systemTheme (useSystemTheme)
```

```
     │                      resolvedTheme = preference === "system" ? systemTheme : preference
     │                      useLayoutEffect → <html data-theme>, useEffect → localStorage, storage
listener
     └── <App>
          ├── <ThemeSwitcher>  useTheme() → preference, setPreference
          └── content          colors from var(--bg), var(--text) ...
```

Start simple. First version: a `dark` boolean in `App` that toggles a class on `<html>`. Then CSS variables, then context so any component can use it, then persistence, System, the no-flash script and sync.

4. Design the state / data model

State	Where	Purpose
`preference`	ThemeProvider (`useState`)	What the user chose: `light` | `dark` | `system` (persisted)
`systemTheme`	`useSystemTheme` (external store)	What the OS prefers: `light` | `dark`
`resolvedTheme`	derived	What we show: `light` | `dark`

```
const [preference, setPreference] = useState(readPreference);      // lazy: read storage once
const systemTheme = useSystemTheme();                               // subscribes to matchMedia
const resolvedTheme = resolveTheme(preference, systemTheme);        // derived, never stored

useLayoutEffect(() => applyTheme(resolvedTheme), [resolvedTheme]);  // <html data-theme>
useEffect(() => savePreference(preference), [preference]);          // localStorage

const value = useMemo(() => ({ preference, resolvedTheme, setPreference }), [preference,
resolvedTheme]);
```

Senior note

- **Choice vs result:** storing only `"dark"` / `"light"` (as the first version did) can't express "follow the OS". Storing the *preference* and deriving the *resolved* theme supports System and keeps one source of truth.
- **The OS setting is external state:** it lives outside React and changes on its own, which is exactly what `useSyncExternalStore` is for (tear-free, works with concurrent rendering, has a server snapshot).
- **Styling lives in CSS, not in React:** React only sets one attribute. Components don't re-render to change colors; CSS variables cascade.
- **Ownership:** `ThemeProvider` owns the preference; `ThemeSwitcher` only calls `setPreference`; everything else reads `resolvedTheme` (e.g. to pick a chart palette).

5. Design the data structures and contracts

Stored value (`localStorage["theme"]`): `"light" | "dark" | "system"`. Anything else is ignored.

Design tokens (one set per theme):

```
:root               { color-scheme: light; --bg: #f3f4f6; --surface: #fff;    --text: #111827; --
primary: #4f46e5; }
:root[data-theme="dark"] { color-scheme: dark;  --bg: #0b1020; --surface: #151b2e; --text: #e5e7eb; --
primary: #818cf8; }
```

Contracts

```
resolveTheme(preference, systemTheme) → "light" | "dark"
readPreference(storage?)  → "light" | "dark" | "system"   // never throws
savePreference(preference, storage?)                       // never throws
applyTheme(resolved)      // document.documentElement.dataset.theme = resolved
useSystemTheme()          → "light" | "dark"               // live
useTheme()                → { preference, resolvedTheme, setPreference }  // throws outside provider
```

6. Trace the data flow

On page load (OS dark, nothing stored):

```
Browser parses <head> → inline script: localStorage "theme" missing → preference "system"
  ↓ matchMedia(dark).matches = true → <html data-theme="dark">  (BEFORE the first paint)
  ↓ CSS loads → :root[data-theme="dark"] variables → first paint is already dark
  ↓ React boots → ThemeProvider: preference "system", systemTheme "dark" → resolved "dark"
  ↓ useLayoutEffect sets data-theme="dark" (no change) → no flash, no flicker
```

On click "Light":

```
ThemeSwitcher onChange → setPreference("light")
  ↓ resolvedTheme = "light" → useLayoutEffect: data-theme="light" (before paint)
  ↓ CSS variables switch → whole page recolors (0.2 s transition)
  ↓ useEffect: localStorage "theme" = "light" → other tabs get a "storage" event → they switch too
```

7. Important technical challenges

Challenge 1: the flash of the wrong theme (FOUC)

- *Problem:* React reads localStorage in `useState` and applies the class in `useEffect`. But the browser paints the default (light) HTML *before* the JS bundle loads, so a dark-mode user sees a white flash on every reload, especially on slow connections.
- *Solution:* A tiny blocking inline script in `<head>` sets `data-theme` before anything paints. React then only keeps it in sync.

```
<script>
  (function () {
    var preference = "system";
    try { var s = localStorage.getItem("theme"); if (s === "light" || s === "dark" || s === "system")
```

```
preference = s; } catch (e) {}
    var dark = preference === "dark" || (preference === "system" && matchMedia("(prefers-color-scheme:
dark)").matches);
    document.documentElement.dataset.theme = dark ? "dark" : "light";
  })();
</script>
```

Challenge 2: "System" and live OS changes

- *Problem:* With only light/dark, a user who uses OS auto-switching gets stuck. Reading `matchMedia(...).matches` once at startup misses later changes, and listening with `useEffect` + `useState` is easy to get wrong (stale listener, missing cleanup).
- *Solution:* Store the preference separately and subscribe to the OS setting with `useSyncExternalStore`.

```
const subscribe = (onChange) => {
  const media = window.matchMedia(QUERY);
  media.addEventListener("change", onChange);
  return () => media.removeEventListener("change", onChange);
};
export const useSystemTheme = () => useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot);
const resolvedTheme = preference === "system" ? systemTheme : preference;
```

Challenge 3: theming with CSS variables (not inline styles)

- *Problem:* `html.dark h1 { color: orange }` rules for every element don't scale: each new component needs dark overrides, and native inputs/scrollbars stay light (white input in a dark page).
- *Solution:* Semantic tokens switched by one attribute; `color-scheme` tells the browser to theme native UI.

```
:root { color-scheme: light; --bg: #f3f4f6; --text: #111827; }
:root[data-theme="dark"] { color-scheme: dark; --bg: #0b1020; --text: #e5e7eb; }
body { background: var(--bg); color: var(--text); }
```

Challenge 4: a safe, efficient context

- *Problem:* `createContext()` with no default + `useContext` outside the provider silently returns `undefined` ("cannot destructure theme of undefined" later). A new `{ theme, toggleTheme }` object every render re-renders every consumer. And `useEffect` applies the theme *after* paint, so for one frame the switcher shows "Dark" while the page is still light.
- *Solution:* A `null` default + a `useTheme` hook that throws a clear error, a memoised value, and `useLayoutEffect` for the DOM attribute.

```
export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error("useTheme must be used inside <ThemeProvider>");
  return context;
}
```

```
const value = useMemo(() => ({ preference, resolvedTheme, setPreference }), [preference,
resolvedTheme]);
useLayoutEffect(() => applyTheme(resolvedTheme), [resolvedTheme]);
```

8. Implement in the correct order

#	Step	Working result
1	Light/dark CSS variables + `data-theme` on `<html>`; a button toggling it	Visible switch
2	Replace hard-coded colors with tokens everywhere; add `color-scheme`	Whole page themed
3	`ThemeContext` + `ThemeProvider` + `useTheme` (throws outside provider)	Theme anywhere
4	Persist: lazy `readPreference` + save effect, validation	Survives reload
5	Preference vs resolved; `useSystemTheme` with `useSyncExternalStore`	System option, live
6	Three-option radio group (`ThemeSwitcher`) with icons + labels	Accessible control
7	Inline `<head>` script mirroring the logic	No flash on load
8	`useLayoutEffect` for applying; `useMemo` for the context value	No flicker, fewer renders
9	`storage` event listener	Tabs in sync
10	Transition + `prefers-reduced-motion`; focus styles	Polished
11	Node self-check for `resolveTheme` / `readPreference` / `savePreference`	Logic proven

9. Cover all states and edge cases

State / edge case	Handling
First visit	System; matches the OS from the first paint
Stored light/dark	Used regardless of the OS
Stored "system"	Follows the OS live
Invalid stored value	System
Storage blocked / full	Works in memory; not saved
JS disabled or slow bundle	Inline script still applies the theme; CSS default is light
OS changes while on System	Live update (`matchMedia` change)
OS changes while on Light/Dark	Ignored (explicit choice wins)
Another tab changes the theme	Synced via the `storage` event
Another tab clears storage	Back to System
`useTheme` outside the provider	Clear error message
Native controls / scrollbars	Follow `color-scheme`
Reduced motion	No color transition

State / edge case	Handling
SSR (no `window`)	`getServerSnapshot` returns "light"; the inline script fixes the first paint

10. Final structure

```
_10_theme_toggle_dark_light_mode/
├── index.html                      ← inline no-flash script in <head>
└── src/
    ├── main.jsx                    ← <ThemeProvider> around <App>
    ├── index.css                   ← tokens for light + dark, color-scheme, body
    ├── App.jsx                     ← demo page using useTheme
    ├── App.css                     ← components styled only with var(--…)
    ├── utils/
    │   └── theme.js                ← resolve / read / save / apply
    ├── hooks/
    │   ├── useSystemTheme.js       ← matchMedia via useSyncExternalStore
    │   └── useTheme.js             ← context consumer with a guard
    ├── context/
    │   ├── themeContext.js         ← createContext(null)
    │   └── ThemeProvider.jsx       ← preference state, apply, save, sync
    └── components/
        └── ThemeSwitcher.jsx       ← Light / Dark / System radio group
```

11. Full working code

`index.html`

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>10-theam-toggle-dark-light-mode</title>
    <!-- Runs before CSS paints and before React loads, so the page never flashes the wrong theme.
         Keep it in sync with src/utils/theme.js (same key, same values). -->
    <script>
      (function () {
        var preference = "system";
        try {
          var stored = localStorage.getItem("theme");
          if (stored === "light" || stored === "dark" || stored === "system") preference = stored;
        } catch (e) {}
        var dark = preference === "dark" ||
          (preference === "system" && window.matchMedia("(prefers-color-scheme: dark)").matches);
```

```
        document.documentElement.dataset.theme = dark ? "dark" : "light";
      })();
    </script>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

src/utils/theme.js

```
export const STORAGE_KEY = "theme";
export const PREFERENCES = ["light", "dark", "system"];

// What the user chose -> what we actually show.
export const resolveTheme = (preference, systemTheme) => (preference === "system" ? systemTheme :
preference);

export function readPreference(storage = globalThis.localStorage) {
  try {
    const value = storage.getItem(STORAGE_KEY);
    return PREFERENCES.includes(value) ? value : "system"; // unknown / missing -> follow the OS
  } catch {
    return "system"; // storage blocked
  }
}

export function savePreference(preference, storage = globalThis.localStorage) {
  try {
    storage.setItem(STORAGE_KEY, preference);
  } catch {
    // storage blocked or full: the theme still works for this visit
  }
}

// One attribute on <html> switches every CSS variable at once.
export function applyTheme(resolvedTheme) {
  document.documentElement.dataset.theme = resolvedTheme;
}
```

src/hooks/useSystemTheme.js

```
import { useSyncExternalStore } from "react";

const QUERY = "(prefers-color-scheme: dark)";
```

```
const subscribe = (onChange) => {
  const media = window.matchMedia(QUERY);
  media.addEventListener("change", onChange);
  return () => media.removeEventListener("change", onChange);
};
const getSnapshot = () => (window.matchMedia(QUERY).matches ? "dark" : "light");
const getServerSnapshot = () => "light"; // no window during SSR

// The OS theme as React state; re-renders when the user switches their OS setting.
export const useSystemTheme = () => useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot);
```

src/context/themeContext.js

```
import { createContext } from "react";

// null default: useTheme() can tell when it is used outside <ThemeProvider>.
export const ThemeContext = createContext(null);
```

src/context/ThemeProvider.jsx

```
import { useEffect, useLayoutEffect, useMemo, useState } from "react";
import { ThemeContext } from "./themeContext";
import { useSystemTheme } from "../hooks/useSystemTheme";
import { STORAGE_KEY, applyTheme, readPreference, resolveTheme, savePreference } from "../utils/theme";

const ThemeProvider = ({ children }) => {
  const [preference, setPreference] = useState(readPreference); // "light" | "dark" | "system"
  const systemTheme = useSystemTheme();
  const resolvedTheme = resolveTheme(preference, systemTheme); // derived, never stored

  // Layout effect: update <html> before the browser paints, so colors and UI switch in the same frame.
  useLayoutEffect(() => applyTheme(resolvedTheme), [resolvedTheme]);
  useEffect(() => savePreference(preference), [preference]);

  // Another tab changed the theme -> follow it.
  useEffect(() => {
    const onStorage = (e) => {
      if (e.key === STORAGE_KEY || e.key === null) setPreference(readPreference());
    };
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, []);

  // Memoised so consumers only re-render when the theme actually changes.
  const value = useMemo(() => ({ preference, resolvedTheme, setPreference }), [preference,
```

```
resolvedTheme]);

  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
};

export default ThemeProvider;
```

src/hooks/useTheme.js

```
import { useContext } from "react";
import { ThemeContext } from "../context/themeContext";

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error("useTheme must be used inside <ThemeProvider>");
  return context;
}
```

src/components/ThemeSwitcher.jsx

```
import { MdComputer, MdDarkMode, MdLightMode } from "react-icons/md";
import { useTheme } from "../hooks/useTheme";

const OPTIONS = [
  { value: "light", label: "Light", icon: <MdLightMode aria-hidden="true" /> },
  { value: "dark", label: "Dark", icon: <MdDarkMode aria-hidden="true" /> },
  { value: "system", label: "System", icon: <MdComputer aria-hidden="true" /> },
];

// A radio group: one choice out of three, arrow keys work, screen readers announce "Dark, selected".
const ThemeSwitcher = () => {
  const { preference, setPreference } = useTheme();

  return (
    <fieldset className="theme-switcher">
      <legend className="sr-only">Theme</legend>
      {OPTIONS.map(({ value, label, icon }) => (
        <label key={value} className={preference === value ? "selected" : ""} title={`${label} theme`}>
          <input
            type="radio"
            name="theme"
            value={value}
            checked={preference === value}
            onChange={() => setPreference(value)}
          />
          {icon}
```

```
          <span>{label}</span>
        </label>
      ))}
    </fieldset>
  );
};

export default ThemeSwitcher;
```

src/App.jsx

```
import "./App.css";
import ThemeSwitcher from "./components/ThemeSwitcher";
import { useTheme } from "./hooks/useTheme";

function App() {
  const { preference, resolvedTheme } = useTheme();

  return (
    <>
      <header className="topbar">
        <strong>Theme Toggle</strong>
        <ThemeSwitcher />
      </header>

      <main className="container">
        <h1>Dark / Light Mode</h1>
        <p aria-live="polite" className="status">
          Showing <strong>{resolvedTheme}</strong> theme
          {preference === "system" && " (following your system setting)"}.
        </p>

        <section className="card">
          <h2>Everything uses CSS variables</h2>
          <p>
            Text, surfaces, borders and links read from <code>var(--…)</code>, so switching one
attribute on{" "}
            <code>&lt;html&gt;</code> recolors the whole page. Native controls follow <code>color-
scheme</code>.
          </p>
          <label htmlFor="demo-input">Native input</label>
          <input id="demo-input" placeholder="Type here" />
          <div className="row">
            <button type="button">Primary button</button>
            <a href="#top">A link</a>
          </div>
        </section>
```

```
      </main>
    </>
  );
}

export default App;
```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";
import ThemeProvider from "./context/ThemeProvider.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </StrictMode>
);
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Design tokens: components only use these variables, never raw colors. */
:root {
  color-scheme: light; /* native inputs, scrollbars and form controls match */
  --bg: #f3f4f6;
  --surface: #ffffff;
  --text: #111827;
  --muted: #4b5563;
  --border: #e5e7eb;
  --primary: #4f46e5;
  --primary-text: #ffffff;
  --link: #4338ca;
}

:root[data-theme="dark"] {
  color-scheme: dark;
```

```
  --bg: #0b1020;
  --surface: #151b2e;
  --text: #e5e7eb;
  --muted: #9ca3af;
  --border: #2a3350;
  --primary: #818cf8;
  --primary-text: #0b1020;
  --link: #a5b4fc;
}

body {
  background: var(--bg);
  color: var(--text);
  font-family: system-ui, sans-serif;
  transition: background-color 0.2s ease, color 0.2s ease;
}

@media (prefers-reduced-motion: reduce) {
  body { transition: none; }
}
```

src/App.css

```
.topbar { display: flex; justify-content: space-between; align-items: center; gap: 12px; flex-wrap:
wrap; padding: 12px 20px; background: var(--surface); border-bottom: 1px solid var(--border); }
.container { max-width: 640px; margin: 32px auto; padding: 0 16px; }
h1 { font-size: 32px; margin-bottom: 8px; }
.status { color: var(--muted); margin-bottom: 20px; }

.theme-switcher { display: flex; gap: 4px; padding: 4px; border: 1px solid var(--border); border-radius:
999px; background: var(--bg); }
.theme-switcher label { display: flex; align-items: center; gap: 6px; padding: 6px 12px; border-radius:
999px; cursor: pointer; color: var(--muted); font-size: 14px; }
.theme-switcher label.selected { background: var(--primary); color: var(--primary-text); }
.theme-switcher label:has(input:focus-visible) { outline: 2px solid var(--primary); outline-offset: 2px;
}
.theme-switcher input { position: absolute; opacity: 0; width: 0; height: 0; }

.card { background: var(--surface); border: 1px solid var(--border); border-radius: 12px; padding: 20px;
display: flex; flex-direction: column; gap: 10px; }
.card h2 { font-size: 20px; }
.card p { color: var(--muted); line-height: 1.5; }
code { font-family: Consolas, monospace; background: var(--bg); padding: 1px 4px; border-radius: 4px; }
.card label { font-weight: 600; }
.card input { padding: 8px 10px; border: 1px solid var(--border); border-radius: 6px; font-size: 15px; }
.row { display: flex; gap: 16px; align-items: center; }
button { padding: 8px 16px; border: none; border-radius: 8px; background: var(--primary); color: var(--
```

```
primary-text); font-size: 15px; cursor: pointer; }
a { color: var(--link); }

.sr-only { position: absolute; width: 1px; height: 1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; }
```

12. Wrap-up

Edge cases handled: no flash on load, System option with live OS changes, explicit choices override the OS, invalid or blocked storage, cross-tab sync (including clear), native controls themed, context used outside the provider, no one-frame mismatch on switch, reduced motion, SSR-safe snapshot.

Trade-offs to mention to the interviewer

- `data-theme` *attribute vs* `.dark` *class:* both work. An attribute can hold more values (`data-theme="high-contrast"`) and doesn't clash with other classes; Tailwind's `dark:` variant expects a class by default (configurable).
- *Inline script vs no script:* the script duplicates a little logic (keep it tiny and in sync) and needs a CSP allowance (hash or nonce), but it's the only way to avoid the flash in a client-rendered app.
- *localStorage vs cookie:* localStorage is simple, but the server can't read it. With SSR, use a cookie so the server renders the right theme directly.
- *Three options vs a single toggle:* a single icon button is compact, but can't express "follow my system". Many apps show a toggle that cycles or a menu with three options.
- *Transition on theme switch:* looks smooth, but every transitioned property animates at once; some apps disable transitions during the switch to avoid jank.

What to add with more time

- Update `<meta name="theme-color">` so the mobile browser bar matches.
- Theme-aware images (`<picture>` with `media="(prefers-color-scheme: dark)"` doesn't follow a manual choice, so swap the `src` by `resolvedTheme`).
- High-contrast theme and `forced-colors` support.
- Tests: `resolveTheme` / storage units; React Testing Library with a mocked `matchMedia` for System changes; a check that `useTheme` throws outside the provider.
- Save the preference to the user's account for cross-device consistency.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Why offer "System" instead of only light and dark?

Many users set dark mode at the OS level (sometimes on a schedule). "System" respects that without forcing a choice, and it's the best default for new users. An explicit Light/Dark choice then overrides it.

Q2. How do you avoid the flash of the wrong theme on reload?

Apply the theme before the first paint with a small blocking inline script in `<head>` (read the stored preference + `matchMedia` , set `data-theme`). React-based effects run too late, after the HTML has already

painted. With SSR, read a cookie on the server and render the right attribute in the HTML.

Q3. Class on `<html>` vs a context value read by each component: which is better for colors?

The attribute + CSS variables: one DOM change recolors everything through the cascade, with no React re-renders and no prop drilling. Use the context value only when JS needs the theme (chart colors, canvas, picking an image).

State design

Q4. Why store the preference and derive the resolved theme?

Because "system" isn't a color, it's a rule. Storing `"dark"` loses the user's intent to follow the OS. With `preference` stored and `resolvedTheme = preference === "system" ? systemTheme : preference`, both are always consistent.

Q5. Why `useSyncExternalStore` for `matchMedia`?

The OS theme is state outside React that can change any time. `useSyncExternalStore` subscribes correctly, avoids tearing in concurrent rendering, re-renders only when the snapshot changes, and supports a server snapshot. A `useState` + `useEffect` listener works too, but it's easier to get wrong.

Q6. Why `useLayoutEffect` to apply the theme?

It runs after React updates the DOM but before the browser paints, so the switcher's new state and the new colors appear in the same frame. With `useEffect`, there can be one frame where the radio says "Dark" but the page is still light.

Q7. Why does the context default to `null`, and why throw in `useTheme`?

With `createContext()` and no provider, `useContext` returns `undefined`, and the error appears somewhere else ("cannot destructure property theme of undefined"). Throwing a clear message in the hook points straight at the missing provider.

Hard parts

Q8. What does `color-scheme` do?

It tells the browser which color schemes the page supports, so built-in UI (form controls, scrollbars, `<select>` dropdowns, the default canvas background) uses dark or light styling. Without it, you get bright white inputs inside a dark page.

Q9. How do you keep the inline script and the React code in sync?

Use the same key and the same allowed values, keep the script minimal (no imports), and comment both places. Some setups generate the script from the shared constants at build time. Test both paths: a fresh load (script) and a toggle (React).

Q10. Why not put `prefers-color-scheme` media queries directly in CSS?

That works for "follow the OS" only. A manual Light/Dark choice can't override a media query. So the media query decides only when the preference is System, and the attribute is the single thing CSS reads.

Q11. How do theme-specific images work?

`<picture><source media="(prefers-color-scheme: dark)">` follows the OS, not the user's manual choice. For a manual theme, choose the image `src` from `resolvedTheme`, use CSS `background-image` per `data-theme`, or use SVGs with `currentColor` so they inherit the text color.

Implementation steps

Q12. How would you test the theme logic?

Unit-test `resolveTheme` and storage helpers (valid, invalid, blocked storage). In React Testing Library, mock `window.matchMedia` (jsdom doesn't implement it) with a controllable `matches` and dispatch `change` to test System. Assert `document.documentElement.dataset.theme` after clicking options, and that `useTheme` throws outside the provider.

Q13. The switcher uses radio buttons. Why not three buttons or a toggle?

It's a single choice from a set, which is exactly what a radio group expresses. You get arrow-key navigation and "selected" announcements for free. A toggle button (`aria-pressed`) fits only a two-state switch.

Q14. Your CSP blocks inline scripts. Now what?

Allow that one script with a hash (`script-src 'sha256-…'`) or a per-request nonce. Or move it to a tiny external file loaded synchronously in `<head>` (a small extra request, still blocking, still before paint).

Performance and scale

Q15. Does switching the theme re-render the whole React app?

Only the context consumers re-render (the memoised value changes), which here is the switcher and the status text. The recoloring itself is done by CSS variables, with no React work. Components that don't call `useTheme` don't re-render.

Q16. Switching causes a janky animation on a big page. What do you do?

Transition only a few properties (background-color, color), or temporarily disable all transitions during the switch (add a `.no-transitions` class, switch, remove it on the next frame). Also respect `prefers-reduced-motion`. The View Transitions API can cross-fade the whole page cheaply.

Q17. How would you support 10 brand themes?

Keep semantic tokens (`--bg`, `--text`, `--primary`) and add one CSS block per theme (`[data-theme="ocean"]`), generated from a design-token file (JSON → CSS). Components never change; only the token values do. Load rarely used theme CSS lazily.

Trade-offs and "why not X?"

Q18. Why not store the theme in a cookie?

With pure client rendering, localStorage is simpler and never sent to the server. With SSR you **should** use a cookie, so the server renders `data-theme` into the HTML and no inline script or flash is needed.

Q19. Why not a library like `next-themes`?

In Next.js, `next-themes` already does exactly this (system option, no-flash script, storage, sync, SSR). In an interview, building it shows you understand FOUC, `matchMedia`, CSS variables and context.

Q20. Why not Tailwind's `dark:` classes instead of CSS variables?

Tailwind's `dark:` variant works well (configure it to use a class or attribute). But every element needs paired classes (`bg-white dark:bg-gray-900`). Semantic CSS variables (which Tailwind can also use) keep components theme-agnostic and make extra themes easy.

Q21. What accessibility issues are specific to dark mode?

Contrast must pass WCAG in **both** themes (pure white on pure black can cause halation; use off-white on dark gray). Focus rings and error colors must stay visible. Don't rely on color alone, and support `forced-colors` (Windows high contrast) so your custom colors don't hide UI.